

# Le superfici con OpenGL

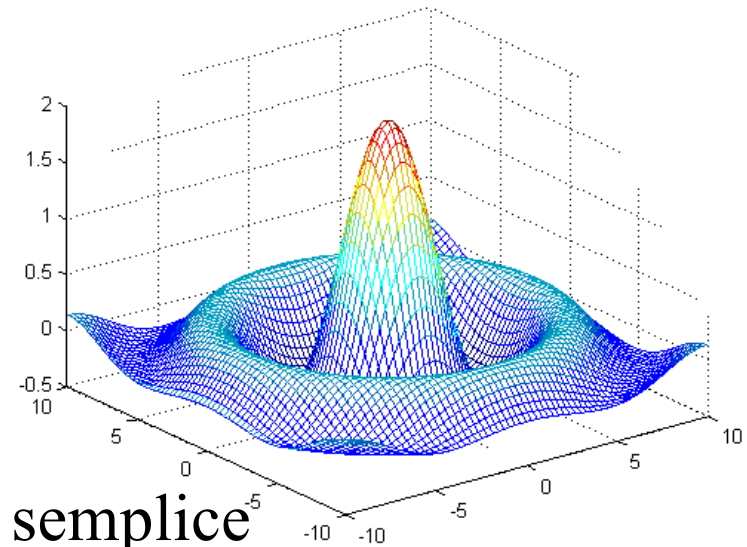
Computer Graphics 2025/2026

# Rappresentazione non parametrica esplicita

$$z = f(x, y) \quad (x, y) \in [a, b] \times [c, d]$$

## Esempio

$$z = \frac{2\sin(\sqrt{x^2 + y^2})}{\sqrt{x^2 + y^2}} \quad (x, y) \in [-10, 10] \times [-10, 10]$$



- Il calcolo di punti sulla superficie è semplice
- Fornisce un unico valore della variabile dipendente per ciascuna coppia di valori delle variabili indipendenti
  - Non consente di rappresentare superfici chiuse o a più valori (es. sfera, ...)

# Rappresentazione non parametrica implicita

$$f(x, y, z) = 0 \quad (x, y, z) \in [a, b] \times [c, d] \times [e, f]$$

## Esempio

Sfera di raggio  $r$  con centro nell'origine

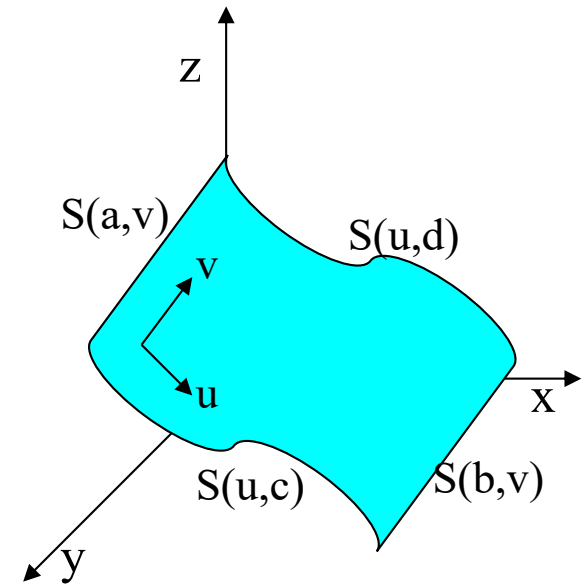
$$x^2 + y^2 + z^2 - r^2 = 0 \quad (x, y, z) \in \mathbb{R}^3$$

- Consente di rappresentare tutte le superfici
- Il calcolo di punti sulla superficie è in genere difficile

# Rappresentazione parametrica

$$\begin{cases} x = x(u, v) \\ y = y(u, v) \\ z = z(u, v) \end{cases} \quad (u, v) \in [a, b] \times [c, d]$$

$$S(u, v) = [x(u, v), y(u, v), z(u, v)]^T$$

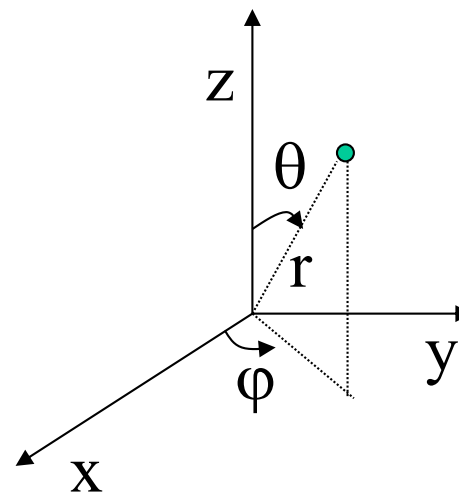


# Rappresentazione parametrica: esempio

Sfera di raggio  $r$  con centro nell'origine

$$\begin{cases} x = x(\theta, \varphi) = r \sin\theta \cos\varphi \\ y = y(\theta, \varphi) = r \sin\theta \sin\varphi \\ z = z(\theta, \varphi) = r \cos\theta \end{cases} \quad (\theta, \varphi) \in [0, \pi] \times [-\pi, \pi]$$

$(\theta, \varphi) = \textit{coordinate sferiche}$



# Rappresentazione parametrica

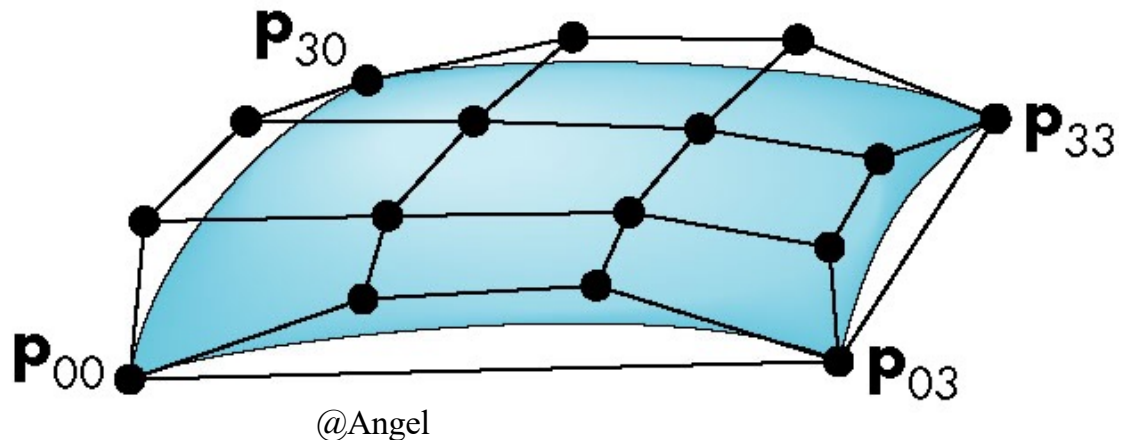
- È esplicita → Il calcolo di punti della superficie è semplice
- Consente di rappresentare superfici chiuse o con valori multipli
- Adatta alla formulazione di superfici composite (unione di più superfici)
- Consente di esprimere una superficie come combinazione lineare di funzioni scalari dei parametri, con coefficienti vettoriali:

$$S(u, v) = \sum_{i=0}^n \sum_{j=0}^m c_{i,j} f_{i,j}(u, v) \quad (u, v) \in [a, b] \times [c, d]$$

# Superfici di Bezier

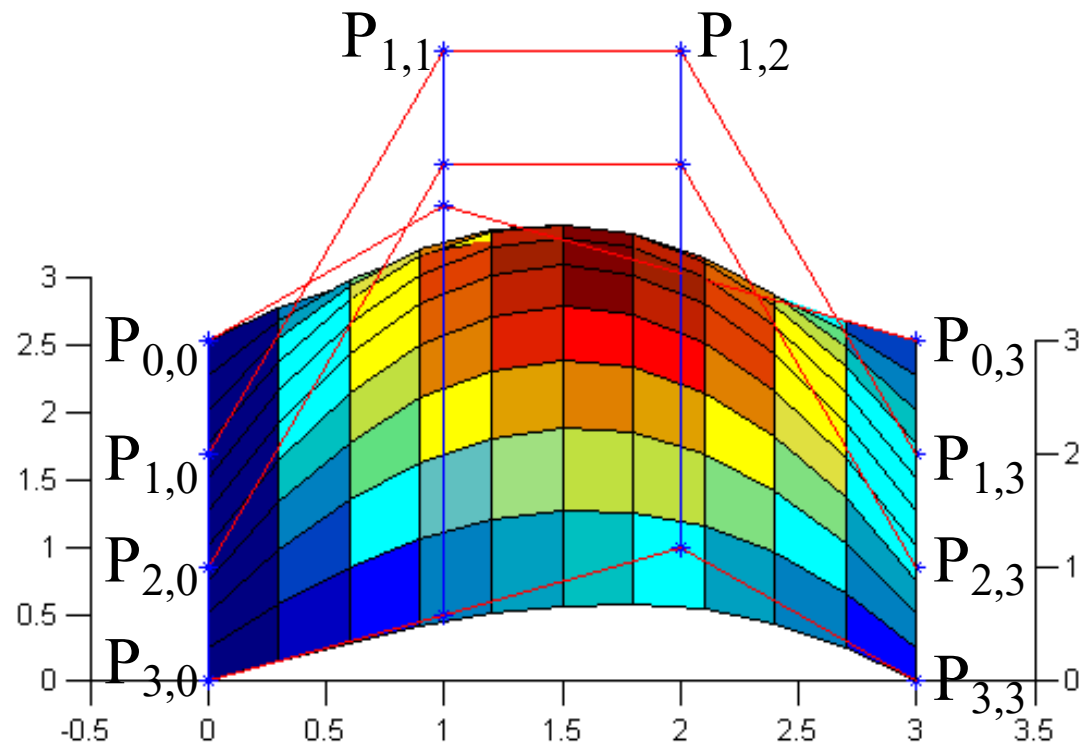
$$S(u, v) = \sum_{i=0}^n \sum_{j=0}^m P_{i,j} B_{i,n}(u) B_{j,m}(v) \quad (u, v) \in [0,1] \times [0,1]$$

- $P_{i,j}$ ,  $i=0, \dots, n$ ,  $j=0, \dots, m$  *punti di controllo* (formano il *poliedro di controllo*)
- $B_{i,n}(u)$ ,  $B_{j,m}(v)$  polinomi di Bernstein di grado  $n$  e  $m$
- $(n,m)$  grado della superficie



# Esempio: superficie di Bezier di grado (3,3)

$$S(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 P_{i,j} B_{i,3}(u) B_{j,3}(v) \quad (u, v) \in [0,1] \times [0,1]$$





# Proprietà delle superfici di Bezier

- *Interpolazione*:  $S(u,v)$  interpola i punti d'angolo del poliedro di controllo:

$$S(0,0)=P_{0,0}, S(1,0)=P_{n,0}, S(0,1)=P_{0,m}, S(1,1)=P_{n,m}$$

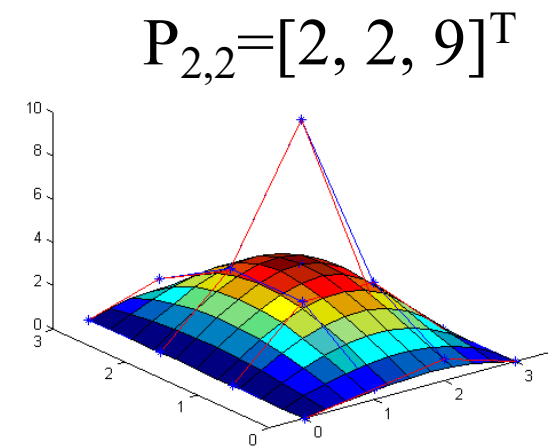
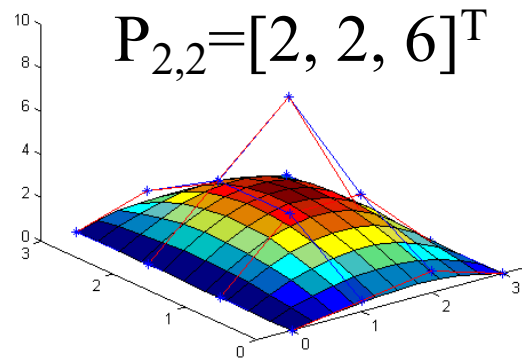
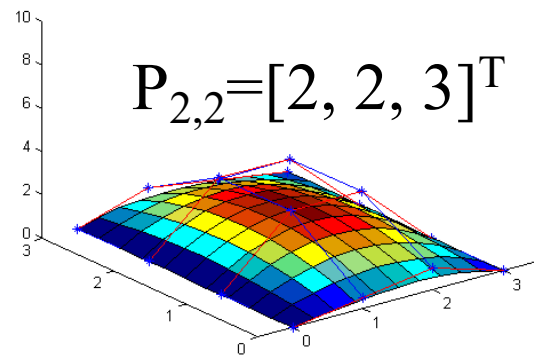
- *Convex hull*: giace completamente all'interno del più piccolo insieme convesso contenente i punti di controllo
- *Invarianza affine*: la relazione fra la superficie e i punti di controllo è invariante per trasformazioni affini
- *Precisione lineare*: se tutti i punti di controllo sono su un piano, la superficie giace sul piano stesso
- Le 4 curve di bordo sono curve di Bezier:

$$S(u,0) = \sum_{i=0}^n P_{i,0} B_{i,n}(u)$$

e analogamente per  $S(u,1)$ ,  $S(0,v)$ ,  $S(1,v)$

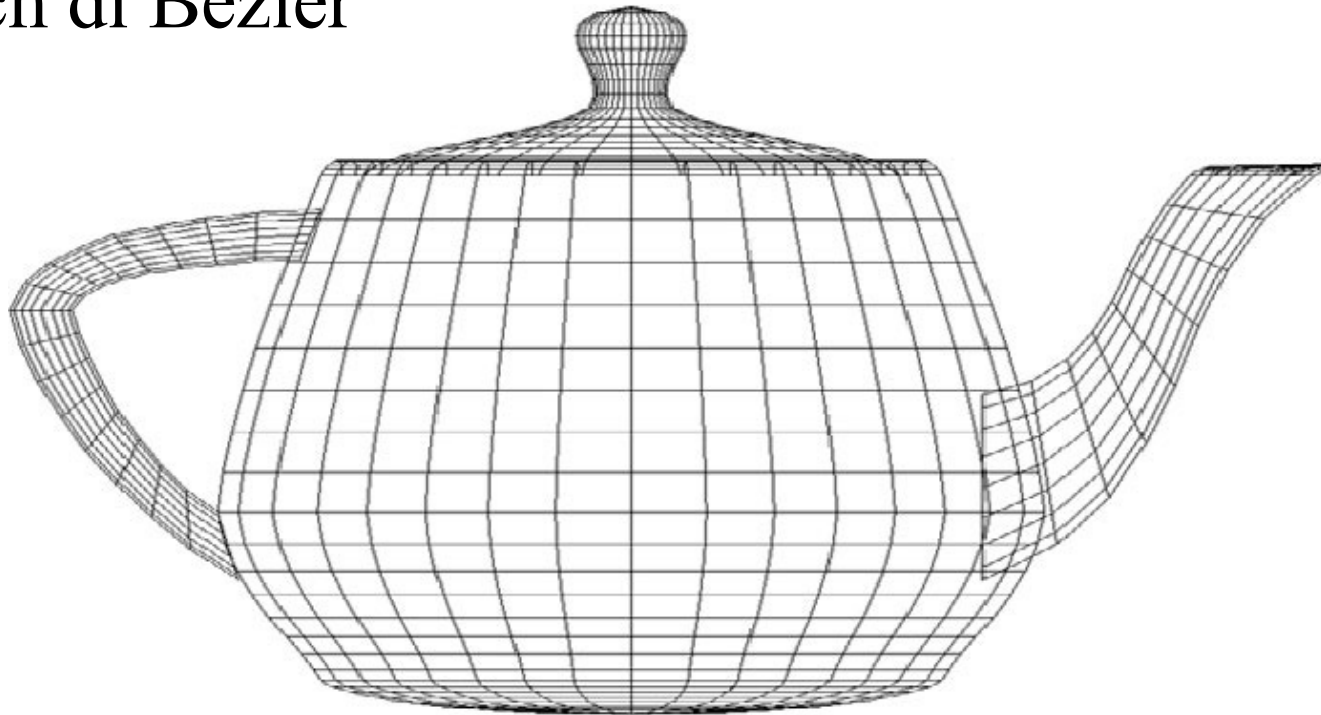
# Proprietà delle superfici di Bezier (cont.)

- *Controllo globale*: spostando un punto di controllo la forma della superficie viene interamente modificata



# Utah teapot

- Uno dei più famosi insiemi di dati in Computer Graphics
- Lista di 306 vertici 3D e indici che definiscono 32 patch di Bezier



# Superfici di Bezier in OpenGL

“Valutatori” 2D, definiti mediante:

```
glMap2f(type, u_min, u_max, u_stride, u_order,  
          v_min, v_max, v_stride, v_order, pts);
```

- type = GL\_MAP2\_VERTEX\_3 (tipo di oggetto da valutare)
- u\_min, u\_max (v\_min, v\_max)= range del parametro u (v)
- u\_stride (v\_stride)= numero di valori f.p. fra un punto di controllo ed il successivo nella direzione u (v)
- u\_order (v\_order) = ordine della curva nella direzione u (v)
- &pts[0][0] = puntatore al primo punto di controllo

## Superfici di Bezier in OpenGL (cont.)

Un valutatore deve essere prima abilitato:

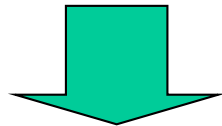
**glEnable(type);**

- **type** = tipo di oggetto da valutare.

Una volta definito ed abilitato un valutatore, per utilizzare un valore dal valutatore attivo si può usare:

**glEvalCoord2f(u,v);**

- **u (v)**= valore del parametro **u (v)** in cui valutare l'oggetto



I vertici della superficie possono essere usati come ogni altro  
vertice

# Superfici di Bezier in OpenGL: esempio

```
GLfloat cps[4][4][3] = {{{-1.5,-1.5,4.0},{-0.5,-1.5,2.0},{0.5,-1.5,-1.0},{1.5,-1.5,2.0}},  
                        {{-1.5,-0.5,1.0},{-0.5,-0.5,3.0},{0.5,-0.5,0.0},{1.5,-0.5,-1.0}},  
                        {{-1.5,0.5,4.0},{-0.5,0.5,0.0},{0.5,0.5,3.0},{1.5,0.5,4.0}},  
                        {{-1.5,1.5,-2.0},{-0.5,1.5,-2.0},{0.5,1.5,0.0},{1.5,1.5,-1.0}}};
```

...

```
glMap2f(GL_MAP2_VERTEX_3, 0, 1, 3, 4, 0, 1, 12, 4, &cps[0][0][0]);  
glEnable(GL_MAP2_VERTEX_3);
```

...

```
for (j = 0; j <= 8; j++) {  
    glBegin(GL_LINE_STRIP);  
        for (i = 0; i <= 30; i++) glEvalCoord2f((GLfloat)i/30.0, (GLfloat)j/8.0);  
    glEnd();  
    glBegin(GL_LINE_STRIP);  
        for (i = 0; i <= 30; i++) glEvalCoord2f((GLfloat)j/8.0, (GLfloat)i/30.0);  
    glEnd();  
}
```

## Superfici di Bezier in OpenGL (cont.)

Se il valutatore attivo deve essere utilizzato per valutare l'oggetto per un insieme di  $n*m$  valori equispaziati dei parametri, si può usare:

```
glMapGrid2f(n, u1, u2, m, v1, v2);
```

```
glEvalMesh2(GL_POINT, i1, i2, j1, j2);
```

```
/* o GL_LINE o GL_FILL*/
```

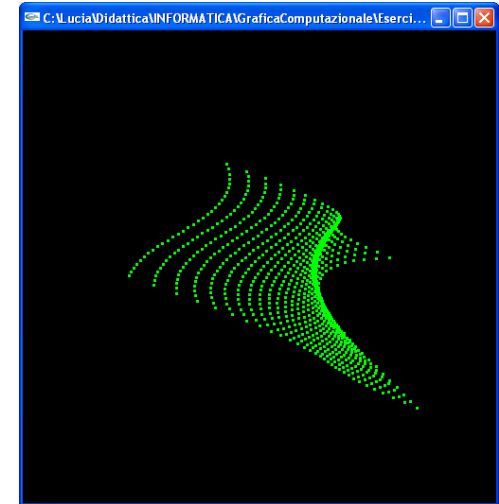
- $0 \leq i1, i2 \leq n$
- $0 \leq j1, j2 \leq m$

# Superfici di Bezier in OpenGL (cont.)

```
glMapGrid2f(n, u1, u2, m, v1, v2);  
glEvalMesh2(GL_POINT, 0, n, 0, m);
```

equivale a:

```
glBegin(GL_POINTS);  
  for (i = 0; i <= n; i++)  
    for (j = 0; j <= m; j++)  
      glEvalCoord2f(u1+i*(u2-u1)/n,  
                   v1+j*(v2-v1)/m);  
glEnd();
```



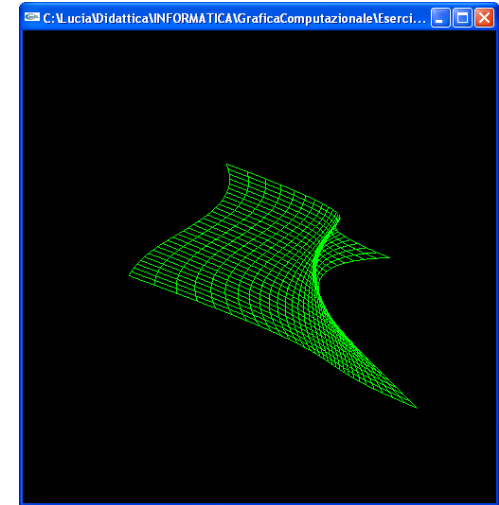


# Superfici di Bezier in OpenGL (cont.)

```
glMapGrid2f(n, u1, u2, m, v1, v2);  
glEvalMesh2(GL_LINE, 0, n, 0, m);
```

equivale a:

```
for (i = 0; i <= n; i++){  
    glBegin(GL_LINE_STRIP);  
        for (j = 0; j <= m; j++) glEvalCoord2f(u1+i*(u2-u1)/n,  
                                                v1+j*(v2-v1)/m);  
    glEnd();}  
for (j = 0; j <= m; j++) {  
    glBegin(GL_LINE_STRIP);  
        for (i = 0; i <= n; i++) glEvalCoord2f(u1+i*(u2-u1)/n,  
                                                v1+j*(v2-v1)/m);  
    glEnd();}
```

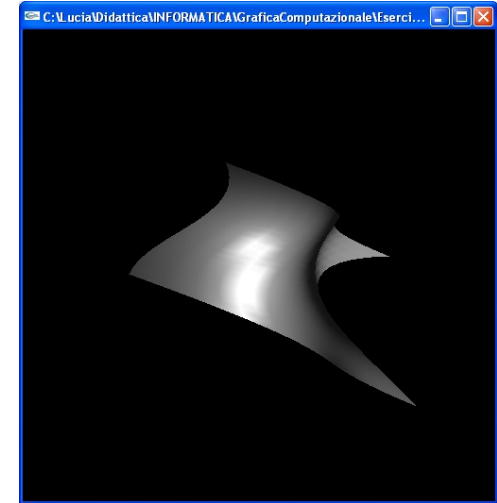


# Superfici di Bezier in OpenGL (cont.)

```
glMapGrid2f(n, u1, u2, m, v1, v2);  
glEvalMesh2(GL_FILL, 0, n, 0, m);
```

equivale a:

```
for (i = 0; i < n; i++){  
    glBegin(GL_QUAD_STRIP);  
    for (j = 0; j <= m; j++) {  
        glEvalCoord2f(u1+i*(u2-u1)/n, v1+j*(v2-v1)/m);  
        glEvalCoord2f(u1+(i+1)*(u2-u1)/n, v1+j*(v2-v1)/m); }  
    glEnd();}
```



## Esercizio 1

- Scrivere un programma (con OpenGL) per il rendering di una superficie di Bezier di ordine (4,4) utilizzando le funzioni di OpenGL

```
GLfloat ctrlpoints[4][4][3] = {  
    {{0, 0, 0},{1, 0, 0.5},{2, 0, 1},{3, 0, 0}},  
    {{0, 1, 0},{1, 1, 3},{2, 1, 3},{3, 1, 0}},  
    {{0, 2, 0},{1, 2, 3},{2, 2, 3},{3, 2, 0}},  
    {{0, 3, 0},{1, 3, 1},{2, 3, 0.5},{3, 3, 0}}};
```

## Suggerimento per GL\_FILL

- Abilitazione del calcolo automatico delle normali:

```
glEnable(GL_AUTO_NORMAL);
```

# Superfici di Bezier razionali

$$S(u, v) = \frac{\sum_{i=0}^n \sum_{j=0}^m w_{i,j} P_{i,j} B_{i,n}(u) B_{j,m}(v)}{\sum_{i=0}^n \sum_{j=0}^m w_{i,j} B_{i,n}(u) B_{j,m}(v)} \quad (u, v) \in [0,1] \times [0,1]$$

## Strategia generale per algoritmi relativi alle superfici razionali

1. Calcolo delle coordinate omogenee  $P_{i,j}^W$  dei punti di controllo  $P_{i,j}$
2. Applicazione dell'algoritmo alla superficie non razionale con punti di controllo  $P_{i,j}^W$
3. Proiezione (omogeneizzazione) del risultato

# Superfici di Bezier razionali in OpenGL: esempio

```
GLfloat cp[4][4][3] = {{{-1.5,-1.5,4.0},{-0.5,-1.5,2.0},{0.5,-1.5,-1.0},{1.5,-1.5,2.0}},  
                        {{-1.5,-0.5,1.0},{-0.5,-0.5,3.0},{0.5,-0.5,0.0},{1.5,-0.5,-1.0}},  
                        {{-1.5,0.5,4.0},{-0.5,0.5,0.0},{0.5,0.5,3.0},{1.5,0.5,4.0}},  
                        {{-1.5,1.5,-2.0},{-0.5,1.5,-2.0},{0.5,1.5,0.0},{1.5,1.5,-1.0}}};
```

```
GLfloat w[4][4]={{1, .5, .5, 1},{1, .5, .5, 1},{1, .5, .5, 1},{1, .5, .5, 1}};
```

```
GLfloat cw[4][4][4];
```

```
...
```

**“Calcolo cw (coordinate omogenee dei cp)”**

```
...
```

```
glMap2f(GL_MAP2_VERTEX_4, 0, 1, 4, 4, 0, 1, 16, 4, &cw[0][0][0]);
```

```
glEnable(GL_MAP2_VERTEX_4);
```

```
...
```

```
glMapGrid2f(30, 0.0, 1.0, 30, 0.0, 1.0);/*inizializza i valori equispaziati*/
```

```
glEvalMesh2(GL_POINT, 0, 30, 0, 30); /* o GL_LINE o GL_FILL */
```

## Esercizio 2

- Scrivere un programma (con OpenGL) per il rendering di una superficie di Bezier razionale di ordine (4,4) utilizzando le funzioni di OpenGL

```
GLfloat ctrlpoints[4][4][3] = {  
    {{0, 0, 0},{1, 0, .5},{2, 0, 1},{3, 0, 0}},  
    {{0, 1, 0},{1, 1, 3},{2, 1, 3},{3, 1, 0}},  
    {{0, 2, 0},{1, 2, 3},{2, 2, 3},{3, 2, 0}},  
    {{0, 3, 0},{1, 3, 1},{2, 3, 0.5},{3, 3, 0}}};
```

```
GLfloat pesi[4][4]={{{1, .5, .5, 1},{1, .5, .5, 1},  
                    {1, .5, .5, 1},{1, .5, .5, 1}}};
```

# Superfici B-Spline

$$S(u, v) = \sum_{i=0}^n \sum_{j=0}^m P_{i,j} N_{i,h}(u) N_{j,k}(v) \quad (u, v) \in [u_h, u_{n+1}] \times [v_k, v_{m+1}]$$

- $P_{i,j}$ ,  $i=0, \dots, n$ ,  $j=0, \dots, m$  punti di controllo
- $N_{i,h}(u)$  e  $N_{j,k}(v)$  funzioni di base B-spline di grado  $h$  e  $k$
- $U=(u_0, u_1, \dots, u_{n+h+1})$  e  $V=(v_0, v_1, \dots, v_{m+k+1})$  vettori dei nodi (non decrescenti)
- $(h,k)$  grado della superficie

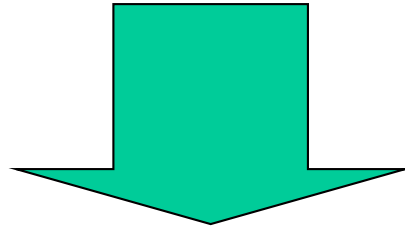


# Proprietà delle superfici B-Spline

- *Interpolazione*:  $S(u,v)$  in genere NON interpola i punti d'angolo del poliedro di controllo
- *Generalizzazione delle superfici di Bezier*
- *Convex hull “ristretto”*: se  $(u,v) \in [u_i, u_{i+1}) * [v_j, v_{j+1})$ ,  $S(u,v)$  giace nel convex hull dei punti di controllo  $P_{i-h,j-k}, \dots, P_{i,j}$
- *Controllo locale*: se  $P_{i,j}$  viene spostato,  $S(u,v)$  risulta modificata solo in corrispondenza degli  $(h+1)*(k+1)$  intervalli di nodi  $[u_i, u_{i+h+1}) * [v_j, v_{j+k+1})$
- $(u,v) \neq (u_i, v_j) \forall i,j \rightarrow S(u,v) \in C^\infty$  nelle direzioni parametriche  $u$  e  $v$
- $u = u_i$  nodo di molteplicità  $q \rightarrow S(u,v) \in C^{h-q}$  nella direzione parametrica  $u$
- *Invarianza affine*: la relazione fra la superficie e i punti di controllo è invariante per trasformazioni affini

## Svantaggio delle superfici B-Spline

Non consentono di rappresentare esattamente alcune forme comuni, quali le quadriche



Superfici NURBS

# Superfici NURBS

$$S(u, v) = \frac{\sum_{i=0}^n \sum_{j=0}^m w_{i,j} P_{i,j} N_{i,h}(u) N_{j,k}(v)}{\sum_{i=0}^n \sum_{j=0}^m w_{i,j} N_{i,h}(u) N_{j,k}(v)} \quad (u, v) \in [u_h, u_{n+1}] \times [v_k, v_{m+1}]$$

- $P_{i,j}$ ,  $i=0, \dots, n$ ,  $j=0, \dots, m$  punti di controllo
- $w_{i,j}$ ,  $i=0, \dots, n$ ,  $j=0, \dots, m$  pesi (non negativi)
- $N_{i,h}(u)$  e  $N_{j,k}(v)$  funzioni di base B-spline di grado  $h$  e  $k$
- $U=(u_0, u_1, \dots, u_{n+h+1})$  e  $V=(v_0, v_1, \dots, v_{m+k+1})$  vettori dei nodi (non decrescenti)
- $(h,k)$  grado della superficie

# Funzioni B-Spline Razionali in due variabili

- Una superficie NURBS può essere riscritta nella forma equivalente:

$$S(u, v) = \sum_{i=0}^m \sum_{j=0}^n P_{i,j} R_{i,j}^{h,k}(u, v)$$

con  $R_{i,j}^{h,k}(t)$  *funzioni B-spline razionali in due variabili* di grado  $(h,k)$  con vettori dei nodi  $U$  e  $V$  e pesi  $w_{i,j}$ :

$$R_{i,j}^{h,k}(u, v) = \frac{w_{i,j} N_{i,h}(u) N_{j,k}(v)}{\sum_{r=0}^m \sum_{s=0}^n w_{r,s} N_{r,h}(u) N_{s,k}(v)}$$

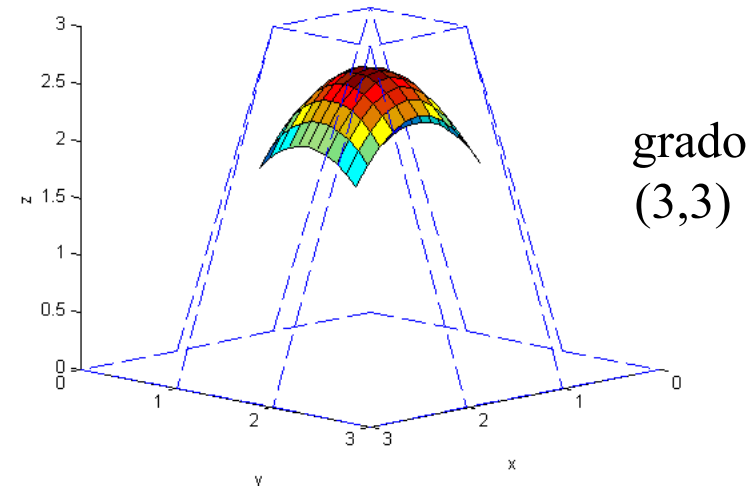
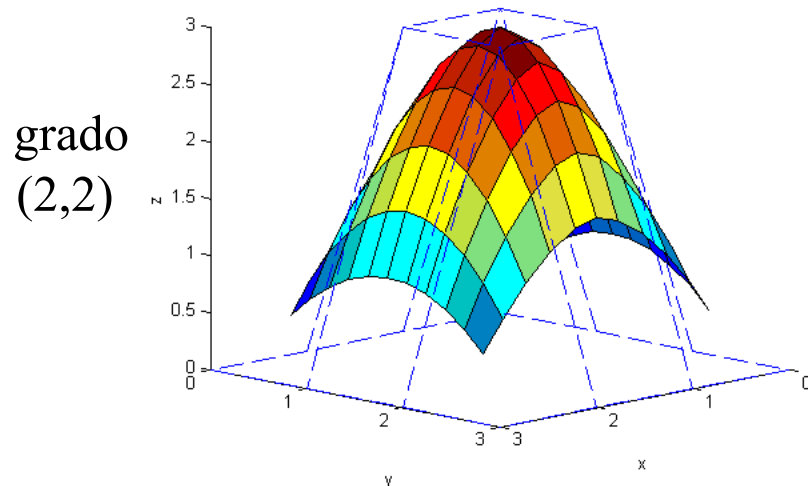
# Proprietà delle superfici NURBS

Analoghe a quelle delle superfici B-Spline:

- *Interpolazione*:  $S(u,v)$  in genere NON interpola i punti d'angolo del poliedro di controllo

U,V uniformi

$$W = \begin{pmatrix} 1 & 0.5 & 0.5 & 1 \\ 1 & 0.5 & 0.5 & 1 \\ 1 & 0.5 & 0.5 & 1 \\ 1 & 0.5 & 0.5 & 1 \end{pmatrix}$$



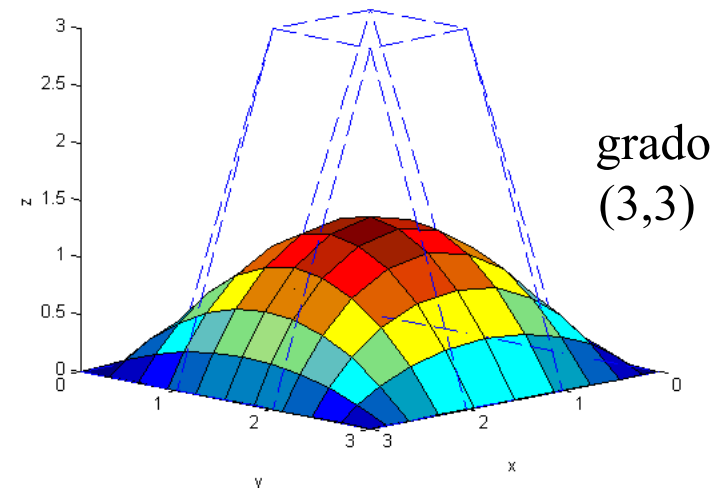
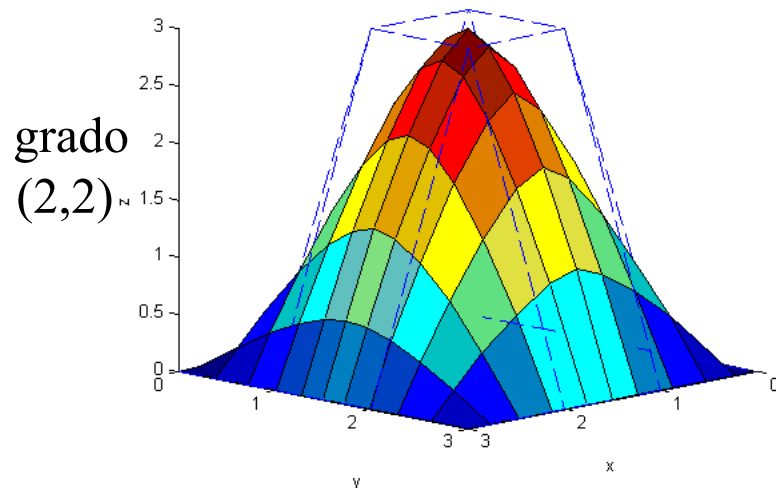
# Proprietà delle superfici NURBS (cont.)

Analoghe a quelle delle superfici B-Spline:

- *Generalizzazione delle superfici di Bezier (e B-Spline)*

$$U=V=(0, 0, 0, 1, 2, 2, 2) \quad W = \begin{pmatrix} 1 & 0.5 & 0.5 & 1 \\ 1 & 0.5 & 0.5 & 1 \\ 1 & 0.5 & 0.5 & 1 \\ 1 & 0.5 & 0.5 & 1 \end{pmatrix}$$

$$U=V=(0, 0, 0, 0, 1, 1, 1, 1)$$



# Proprietà delle superfici NURBS

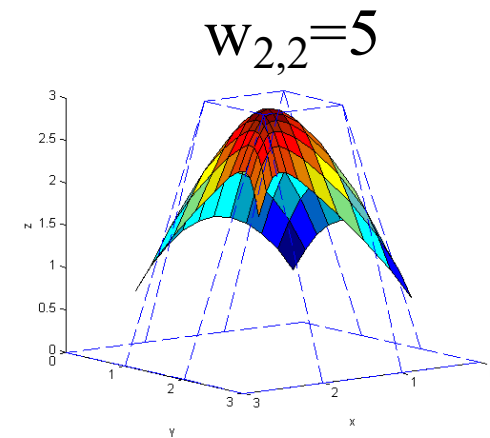
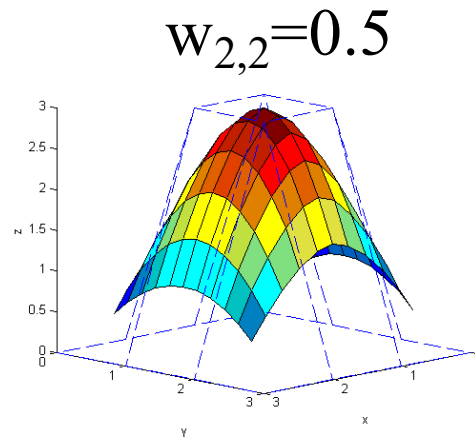
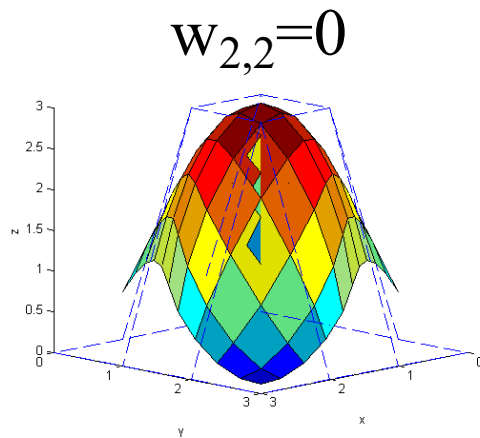
Analoghe a quelle delle superfici B-Spline:

- *Convex hull “ristretto”*: se  $(u,v) \in [u_i, u_{i+1}) * [v_j, v_{j+1})$ ,  $S(u,v)$  giace nel convex hull dei punti di controllo  $P_{i-h,j-k}, \dots, P_{i,j}$
- $(u,v) \neq (u_i, v_j) \forall i,j \rightarrow S(u,v) \in C^\infty$  nelle direzioni parametriche  $u$  e  $v$
- $u = u_i$  nodo di molteplicità  $q \rightarrow S(u,v) \in C^{h-q}$  nella direzione parametrica  $u$

# Proprietà delle superfici NURBS (cont.)

Inoltre:

- *Controllo locale*: se  $P_{i,j}$  viene spostato o  $w_{i,j}$  è modificato,  $S(u,v)$  risulta modificata solo in corrispondenza degli  $(h+1)*(k+1)$  intervalli di nodi  $[u_i, u_{i+h+1})*[v_j, v_{j+k+1})$



- *Invarianza affine e proiettiva*: la relazione fra la superficie e i punti di controllo è invariante per trasformazioni affini e proiettive



# Superfici B-Spline e NURBS in OpenGL

GLU fornisce un insieme di funzioni per la modellazione ed il rendering di superfici B-Spline e NURBS, basate sui valutatori di OpenGL:

**gluNewNurbsRenderer()** come per le curve

**gluBeginSurface()** per iniziare la superficie

**gluNurbsSurface()** per specificare vettore dei nodi, punti di controllo e ordine della superficie (almeno una volta)

**gluEndSurface()** per terminare la superficie

**gluNurbsProperty()** come per le curve

**gluNurbsCallback()** come per le curve

# Superfici B-Spline in OpenGL: esempio

```
GLfloat cp[4][4][3] = {    {{-3, -3, -3}, {-3, -1, -3}, {-3, 1, -3}, {-3, 3, -3}},
                            {{-1, -3, -3}, {-1, -1, 3}, {-1, 1, 3}, {-1, 3, -3}},
                            {{1, -3, -3}, {1, -1, 3}, {1, 1, 3}, {1, 3, -3}},
                            {{3, -3, -3}, {3, -1, -3}, {3, 1, -3}, {3, 3, -3}}};
```

```
GLUnurbsObj *theNurb;
```

```
GLfloat knots[8] = {0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 1.0};
```

```
...
```

```
theNurb=gluNewNurbsRenderer(); /*puntatore all'oggetto NURBS */
```

```
...
```

```
gluBeginSurface(theNurb); /*inizia la superficie*/
```

```
gluNurbsSurface(theNurb, 8, knots, 8, knots 4 * 3, 3 &cp[0][0][0], 4, 4,  
GL_MAP2_VERTEX_3);
```



```
/* vettori dei nodi knots, punti di controllo cp e ordine (4,4) */
```

```
gluEndSurface(theNurb) /* termina la superficie*/
```

# Superfici NURBS in OpenGL: esempio

```
GLfloat cp[4][4][3] = {      {{-3, -3, -3}, {-3, -1, -3}, {-3, 1, -3}, {-3, 3, -3}},
                              {{-1, -3, -3}, {-1, -1, 3}, {-1, 1, 3}, {-1, 3, -3}},
                              {{1, -3, -3}, {1, -1, 3}, {1, 1, 3}, {1, 3, -3}},
                              {{3, -3, -3}, {3, -1, -3}, {3, 1, -3}, {3, 3, -3}}};
```

```
GLUnurbsObj *theNurb;
```

```
GLfloat knots[8] = {0.0, 0.0, 0.0, 0.0, 1.0, 1.0, 1.0, 1.0};
```

```
GLfloat w[4][4]={{1, .5, .5, 1},{1, .5, .5, 1},{1, .5, .5, 1},{1, .5, .5, 1}};
```

```
GLfloat cw[4][4][4];
```

```
...
```

**“Calcolo cw (coordinate omogenee dei cp)”**

```
...
```

```
theNurb=gluNewNurbsRenderer(); /*puntatore all'oggetto NURBS */
```

```
...
```

```
gluBeginSurface(theNurb); /*inizia la superficie*/
```

```
gluNurbsSurface(theNurb,8, knots, 8, knots,4 * 4, 4, &cw[0][0][0],4, 4,  
    GL_MAP2_VERTEX_4);
```

```
/* vettori dei nodi knots, punti di controllo in coord. omog. cw e ordine (4,4) */
```

```
gluEndSurface(theNurb) /* termina la superficie*/
```

# Proprietà delle superfici NURBS in OpenGL

`gluNurbsProperty( GLUnurbsObj *nobj,`  
`Glenum property, Gfloat value)`

sceglie opportuni valori per il rendering

## Esempi:

```
gluNurbsProperty(theNurb, GLU_U_STEP, 5);  
gluNurbsProperty(theNurb, GLU_V_STEP, 10);  
gluNurbsProperty(theNurb, GLU_SAMPLING_METHOD, GLU_DOMAIN_DISTANCE);
```

impone il campionamento della superficie in 5\*10 valori equispaziati dei parametri (u,v)

```
gluNurbsProperty(theNurb, GLU_DISPLAY_MODE, GLU_FILL);
```

visualizzazione mediante poligoni

```
gluNurbsProperty(theNurb, GLU_DISPLAY_MODE, GLU_OUTLINE_POLYGON);
```

visualizzazione mediante contorni dei poligoni (wire-frame)

## Esercizio 3

- Scrivere un programma (con OpenGL) per il rendering di una superficie B-Spline di ordine (4,4) utilizzando le funzioni di OpenGL

```
GLfloat ctlpoints[4][4][3] = {  
    {{-3, -3, -3}, {-3, -1, -3}, {-3, 1, -3}, {-3, 3, -3}},  
    {{-1, -3, -3}, {-1, -1, 3}, {-1, 1, 3}, {-1, 3, -3}},  
    {{1, -3, -3}, {1, -1, 3}, {1, 1, 3}, {1, 3, -3}},  
    {{3, -3, -3}, {3, -1, -3}, {3, 1, -3}, {3, 3, -3}}};
```

## Esercizio 4

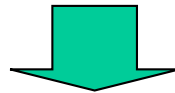
- Scrivere un programma (con OpenGL) per il rendering di una superficie NURBS di ordine (4,4) utilizzando le funzioni di OpenGL

```
GLfloat ctlpoints[4][4][3] = {  
    {{-3, -3, -3}, {-3, -1, -3}, {-3, 1, -3}, {-3, 3, -3}},  
    {{-1, -3, -3}, {-1, -1, 3}, {-1, 1, 3}, {-1, 3, -3}},  
    {{1, -3, -3}, {1, -1, 3}, {1, 1, 3}, {1, 3, -3}},  
    {{3, -3, -3}, {3, -1, -3}, {3, 1, -3}, {3, 3, -3}}};
```

```
GLfloat pesi[4][4]={ {1, .5, .5, 1}, {1, .5, .5, 1},  
                     {1, .5, .5, 1}, {1, .5, .5, 1}};
```

# “Trimming” di superfici NURBS

Le superfici NURBS hanno topologia rettangolare



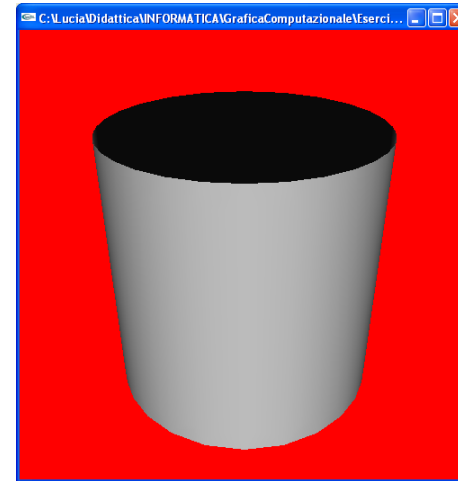
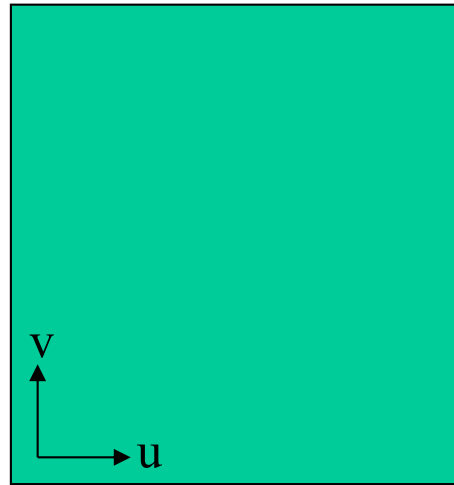
Per avere superfici NURBS con topologia arbitraria si può:

1. ritagliare il dominio parametrico e
2. disegnare la superficie originaria solo nel dominio modificato

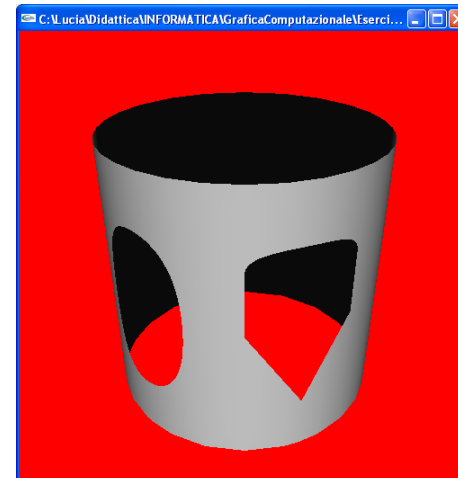
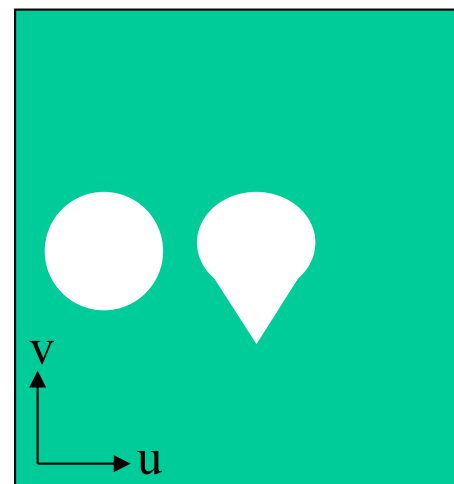
- Applicazioni:
  - Modellazione sculturata/gerarchica
  - Composizione “solida” di superfici mediante operazioni booleane

# Esempio di “trimming”

dominio  
parametrico  
rettangolare



dominio  
parametrico  
rettangolare  
con “buchi”

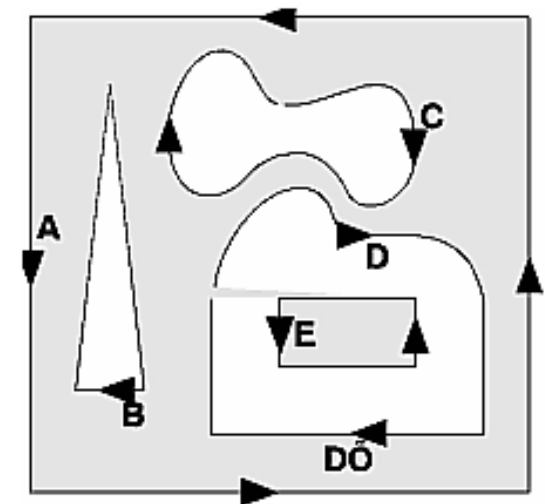




## Trimming di superfici NURBS (cont.)

- Le curve che definiscono le regioni di trimming devono essere:
  - curve chiuse
  - senza intersezioni fra di loro
- L'orientamento delle curve definisce la parte di dominio parametrico da escludere:
  - Orario: interno della curva
  - Antiorario: esterno della curva

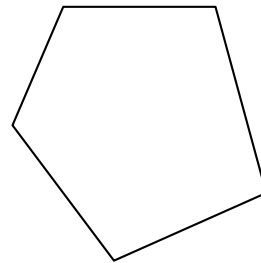
(camminando lungo la curva, tutto ciò che è sulla sinistra viene incluso)



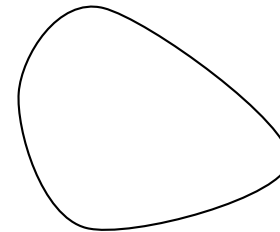
## Trimming di superfici NURBS (cont.)

- Una curva di trimming può essere definita mediante:

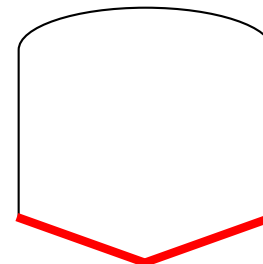
- Una poligonale chiusa



- Una curva NURBS chiusa



- L'unione di una poligonale e di una curva NURBS



# Trimming di superfici NURBS (cont.)

- **Poligonale chiusa:**

```
GLfloat E[5][2]={0,0},{1,0},{1,1},{0,1},{0,0};
```

```
GLfloat P[5][2]={0.3,0.3},{0.3,0.7},  
               {0.7,0.7},{0.7,0.3},{0.3,0.3};
```

...

```
gluBeginSurface(theNurb);
```

```
    gluNurbsSurface(theNurb,10, knotu, 4, knotv,2 * 3, 3, &cp [0][0][0],  
                    3, 2, GL_MAP2_VERTEX_3);
```

```
gluBeginTrim(theNurb); //confine esterno
```

```
    gluPwlCurve(theNurb,5,&E[0][0],2,GLU_MAP1_TRIM_2);
```

```
gluEndTrim(theNurb);
```

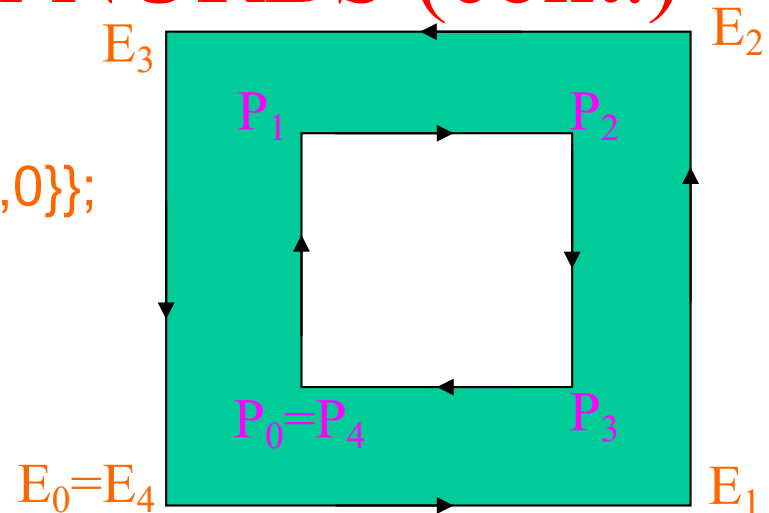
```
gluBeginTrim(theNurb); //confine interno
```

```
    gluPwlCurve(theNurb,5,&P[0][0],2,GLU_MAP1_TRIM_2);
```

```
gluEndTrim(theNurb);
```

```
gluEndSurface(theNurb);
```

...



# Trimming di superfici NURBS (cont.)

- **Curva NURBS chiusa:**

```
GLfloat E[5][2]={{0,0},{1,0},{1,1},{0,1},{0,0}};
```

```
GLfloat P[7][2]={{0.3,0.5},{0.3,0.8},  
                {0.6,0.8},{0.6,0.5},{0.6, 0.2},  
                {0.3, 0.2},{0.3, 0.5}}; ...
```

```
gluBeginSurface(theNurb);
```

```
gluNurbsSurface(theNurb,10, knotu, 4, knotv,2 * 3, 3, &cp [0][0][0],  
                3, 2, GL_MAP2_VERTEX_3);
```

```
gluBeginTrim(theNurb); //confine esterno
```

```
gluPwlCurve(theNurb,5,&E[0][0],2,GLU_MAP1_TRIM_2);
```

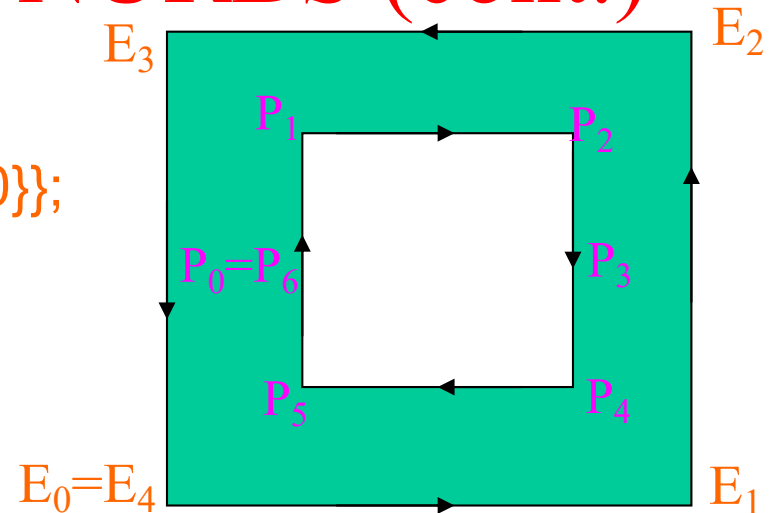
```
gluEndTrim(theNurb);
```

```
gluBeginTrim(theNurb); //confine interno
```

```
gluNurbsCurve(theNurb,10,knotu,2,&P[0][0],3,  
              GLU_MAP1_TRIM_2);
```

```
gluEndTrim(theNurb);
```

```
gluEndSurface(theNurb);
```



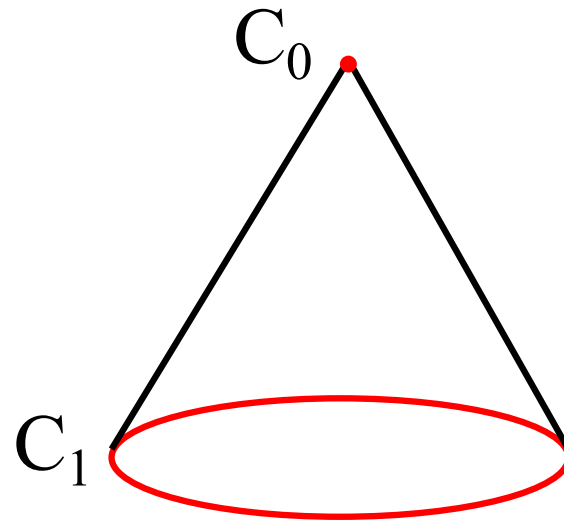
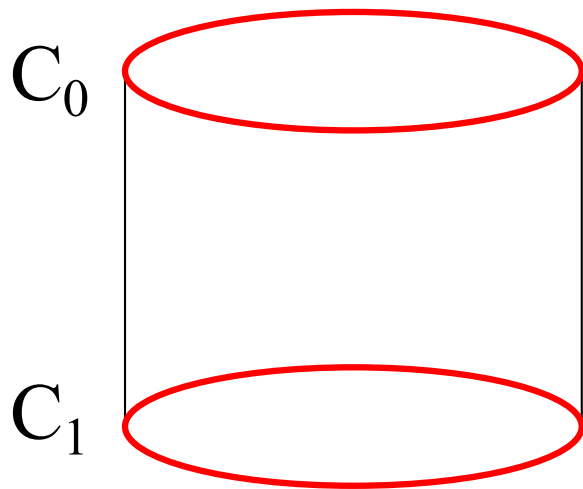
## Esercizio 5

- Scrivere un programma (con OpenGL) per il rendering di una superficie NURBS di ordine (4,4) con trimming utilizzando le funzioni di OpenGL

# Superfici *rigate*

- ottenute mediante interpolazione lineare di due curve di confine note  $C_0$  e  $C_1$

## Esempi



# Superfici *rigate* mediante NURBS

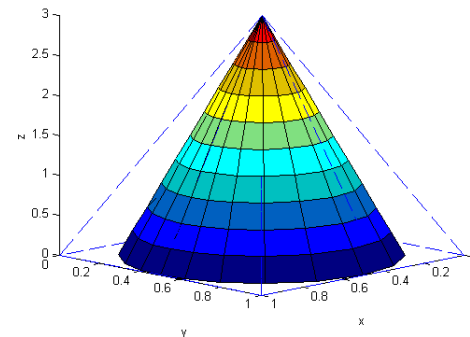
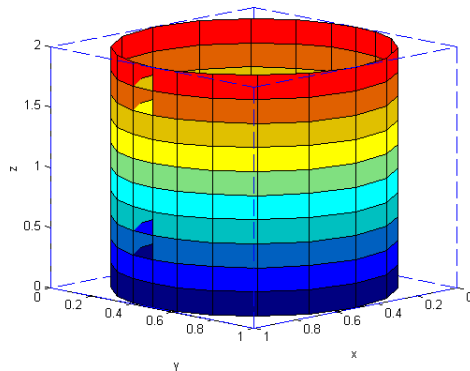
- Se  $C_0$  e  $C_1$  sono rappresentate mediante curve NURBS:

$$C_0(u) = \sum_{i=0}^n P_{i,0} R_{i,h}(u) \quad C_1(u) = \sum_{i=0}^n P_{i,1} R_{i,h}(u)$$

con  $U=(u_0, u_1, \dots, u_{n+h+1})$  e pesi  $w_{i,0}$  e  $w_{i,1}$ , allora la superficie rigata può essere rappresentata mediante la superficie NURBS:

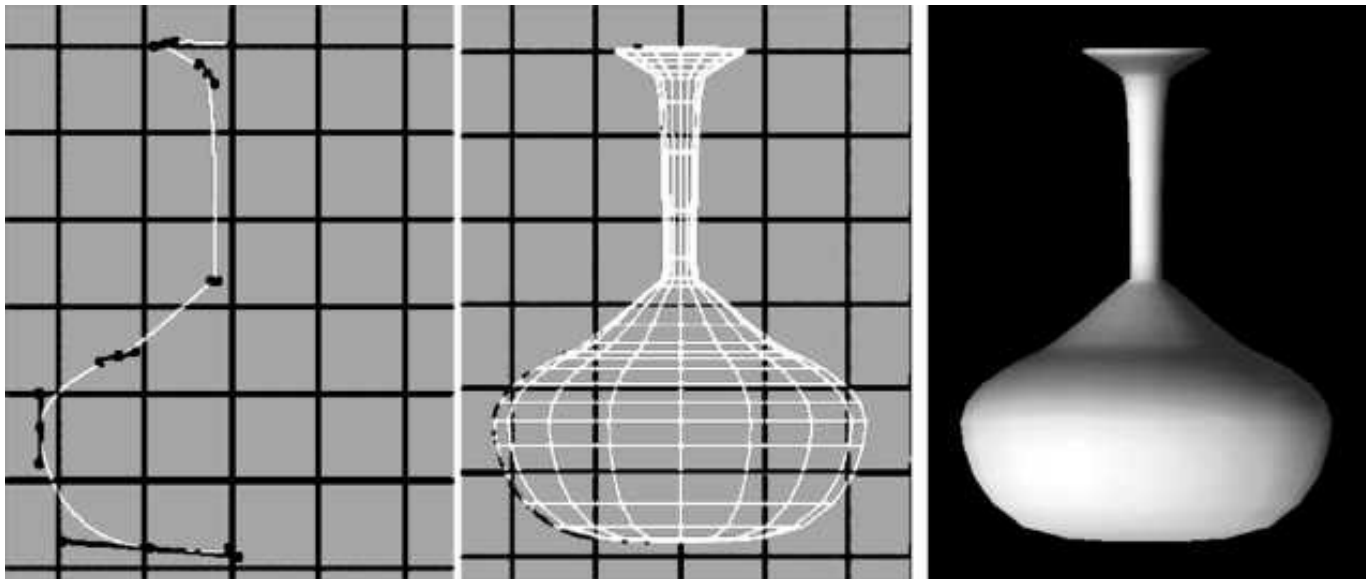
$$S(u, v) = \sum_{i=0}^n \sum_{j=0}^1 P_{i,j} R_{i,j}^{h,k}(u, v)$$

con  $U=(u_0, u_1, \dots, u_{n+h+1})$ ,  $V=(0,0,1,1)$  e pesi  $w_{i,j}$



# Superfici di rotazione (o di rivoluzione)

- ottenute ruotando una curva piana  $C$  (detta *curva generatrice*) intorno ad un asse contenuto nel piano della curva (detto *asse di rotazione*)



@Marini



# Superfici *di rotazione* mediante NURBS

- Se  $C$  è rappresentata mediante la curva NURBS nel piano  $xy$ :

$$C(v) = \sum_{j=0}^n P_j R_{j,k}(v)$$

con  $V=(v_0, v_1, \dots, v_{n+k+1})$  e pesi  $w_j$ , allora la superficie di rotazione può essere rappresentata mediante la superficie NURBS:

$$S(u, v) = \sum_{i=0}^6 \sum_{j=0}^n P_{i,j} R_{i,j}^{2,k}(u, v)$$

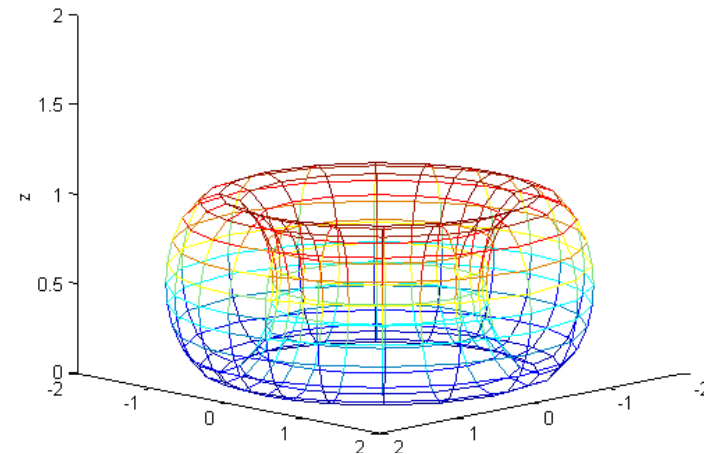
con

- $U=(0,0,0,0.25,0.5,0.5,0.75,1,1,1)$  (quello della circonferenza NURBS);
- $V=(v_0, v_1, \dots, v_{n+k+1})$  (quello di  $C(v)$ );
- $w_{i,j}=(w_j, 0.5 w_j, 0.5 w_j, w_j, 0.5 w_j, 0.5 w_j, w_j)$ ,  $i=0, \dots, 6$ .

# Superfici *di rotazione* mediante NURBS: esempi

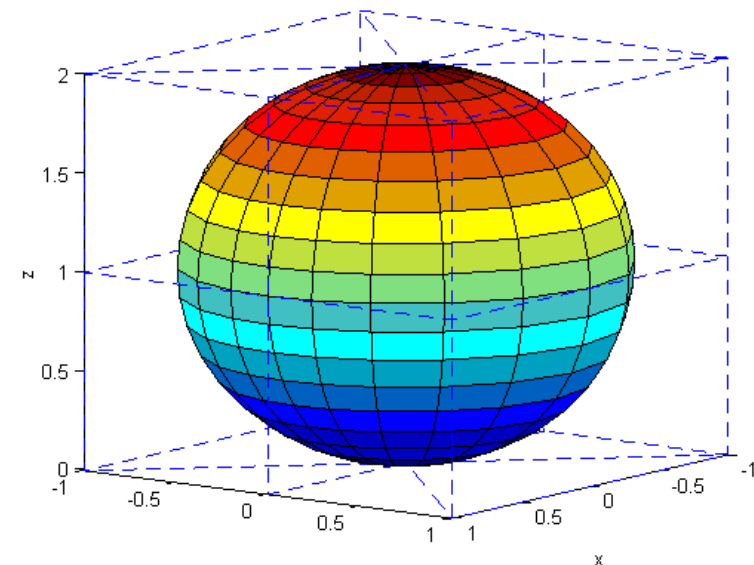
## Toro:

- curva generatrice=circonferenza  
nel piano xz
- asse di rotazione=asse z



## Sfera:

- curva generatrice =  
semicirconferenza  
nel piano xz
- asse di rotazione=asse z



# Altre superfici con GLU

Sfera, cilindro e disco

**Creazione:** `gluNewQuadric()`

**Attributi:**

`gluQuadricOrientation()`, per differenziare l'interno dall'esterno

`gluQuadricDrawStyle()`, per definire se visualizzare come punti, linee o poligoni pieni

`gluQuadricNormals()`, per definire una normale per vertice o per faccia

**Gestione errori:** `gluQuadricCallback()`

**Rendering:**

`gluSphere()`

`gluCylinder()`

`gluDisk()`

`gluPartialDisk()`

**Eliminazione:** `gluDeleteQuadric()`

## Esempio

```
GLUquadricObj *qobj;  
qobj = gluNewQuadric(); //Creazione  
gluQuadricCallback(qobj, GLU_ERROR, (GLvoid  
    (*))errorCallback); //Gestione errori  
gluQuadricDrawStyle(qobj, DrawStyle);  
gluQuadricOrientation(qobj, Orientation);  
gluQuadricNormals(qobj, Normals);  
gluSphere(qobj, 0.75, 15, 10); //raggio, longit., latit.
```

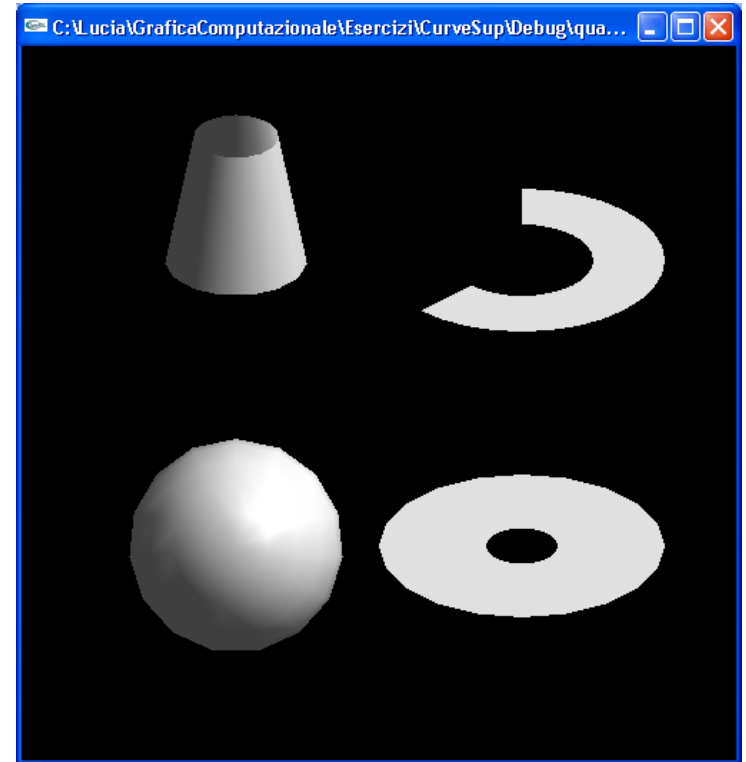
**DrawStyle**= GLU\_FILL, GLU\_LINE, GLU\_SILHOUETTE,  
GLU\_POINT

**Orientation**= GLU\_OUTSIDE, GLU\_INSIDE

**Normals**= GLU\_SMOOTH, GLU\_FLAT, GLU\_NONE

# Sfera, cilindro e disco

```
void gluSphere (GLUquadricObj *qobj,  
               GLdouble radius,  
               GLint slices, GLint stacks);  
void gluCylinder (GLUquadricObj *qobj,  
                GLdouble baseRadius,  
                GLdouble topRadius,  
                GLdouble height,  
                GLint slices, GLint stacks);  
void gluDisk (GLUquadricObj *qobj, GLdouble innerRadius,  
             GLdouble outerRadius, GLint slices, GLint rings);  
void gluPartialDisk (GLUquadricObj *qobj, GLdouble innerRadius,  
                   GLdouble outerRadius, GLint slices, GLint rings,  
                   GLdouble startAngle, GLdouble sweepAngle);
```



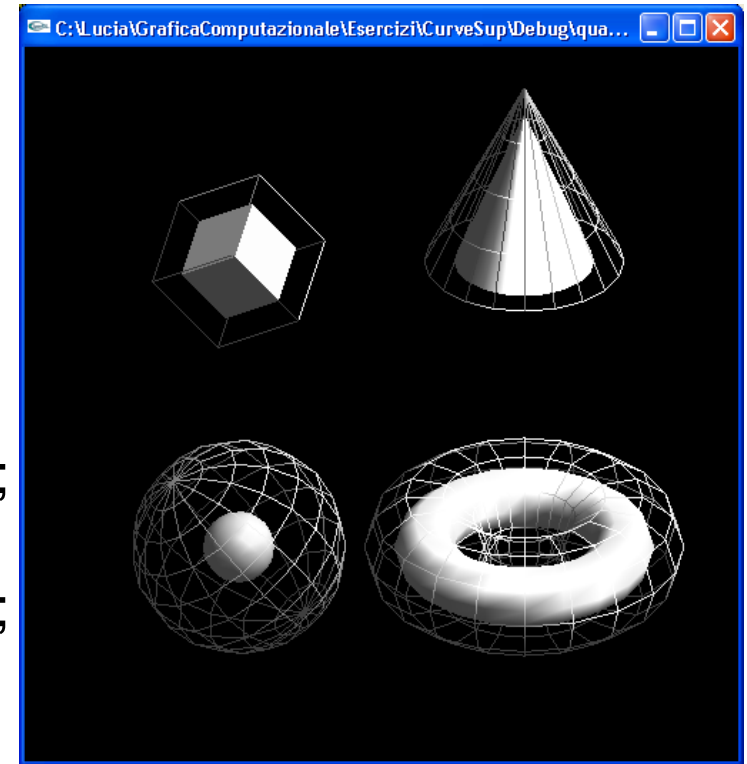
## Esercizio 6

- Scrivere un programma (con OpenGL) per il rendering di sfera, cilindro e dischi utilizzando le funzioni di GLU

# Altre superfici con GLUT

Sfera, cubo, toro, cono

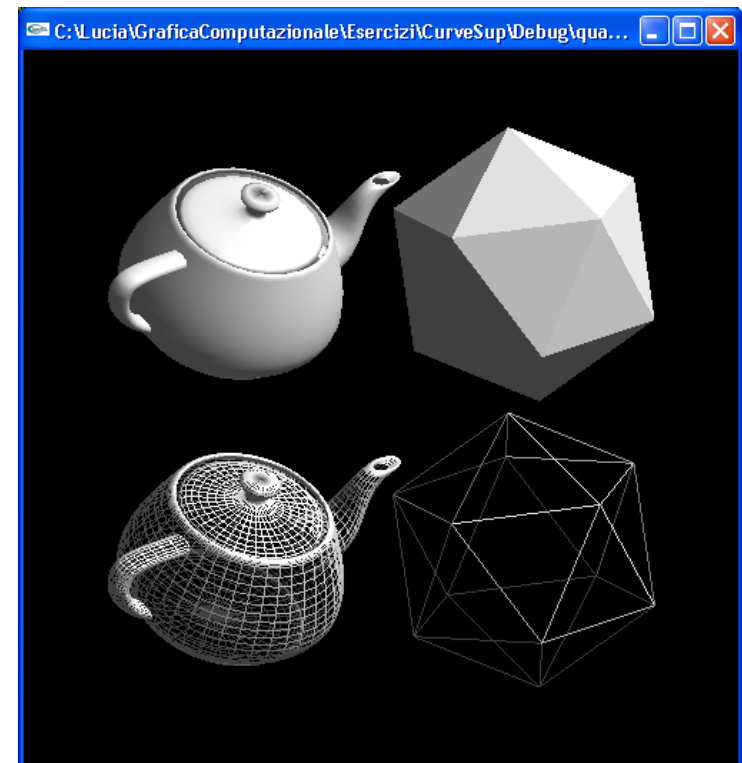
```
void glutWireSphere(GLdouble radius,  
                    GLint slices, GLint stacks);  
void glutSolidSphere(GLdouble radius,  
                     GLint slices, GLint stacks);  
void glutWireCube(GLdouble size);  
void glutSolidCube(GLdouble size);  
void glutWireTorus(GLdouble innerRadius, GLdouble outerRadius,  
                   GLint nsides, GLint rings);  
void glutSolidTorus(GLdouble innerRadius, GLdouble outerRadius,  
                    GLint nsides, GLint rings);  
void glutWireCone(GLdouble radius, GLdouble height,  
                  GLint slices, GLint stacks);  
void glutSolidCone(GLdouble radius, GLdouble height,  
                   GLint slices, GLint stacks);
```



# Altre superfici con GLUT

Teiera, icosaedro

```
void glutWireTeapot(GLdouble size);  
void glutSolidTeapot(GLdouble size);  
void glutWireIcosahedron(void);  
void glutSolidIcosahedron(void);
```

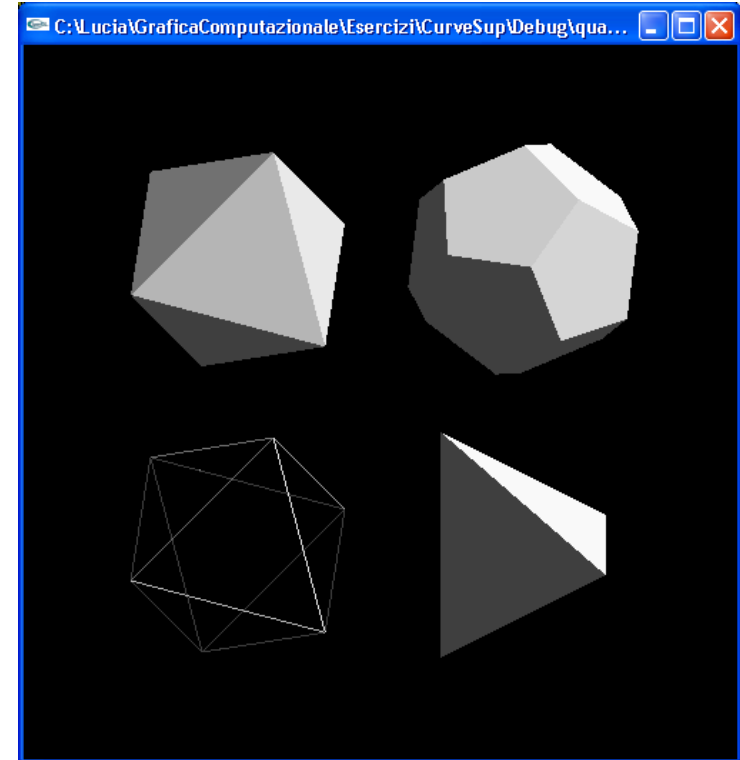




# Altre superfici con GLUT

Ottaedro, tetraedro, dodecaedro

```
void glutWireOctahedron(void);  
void glutSolidOctahedron(void);  
void glutWireTetrahedron(void);  
void glutSolidTetrahedron(void);  
void glutWireDodecahedron(void);  
void glutSolidDodecahedron(void);
```



## Esercizio 7

- Scrivere un programma (con OpenGL) per il rendering di un albero di Natale

## Testo di riferimento per la lezione sulle superfici in OpenGL:

- *OpenGL Programming Guide, II edition*, Addison-Wesley Publishing Company:
  - **Cap. 11 (Tessellators and Quadrics)**
  - **Cap. 12 (Evaluators and NURBS)**
  - **Appendix D (Basics of GLUT)**